

VCU Technical Handbook

Arnav Bala

Contents

Introduction	1
System Architecture	1
Logic Flow	2
Key Implementations	3
CAN Communications	4
Maintenance Notes	4

Introduction

The Vehicular Control Unit (VCU) serves as the processing hub for RBR's car, designed to manage vehicle inputs and outputs in the electric vehicle. Built on ATmega 328P MCU using the Arduino framework, the VCU processes analog signals from accelerator pedal (APPS_5V and APPS_3V3 for redundancy and fault detection) and brake pedal, as well as a digital input from the start button. It facilitates communication over the Controller Area Network (CAN) bus with critical systems, including the Battery Management System (BMS) for readiness checks, the motor controller for torque commands, and a debug interface for monitoring and logging. Non-CAN outputs are limited to buzzer, brake light, and drive mode LED. This implementation emphasizes safety through fault monitoring and a state machine architecture, ensuring reliable operation from initialization to full drive mode.

System Architecture

The VCU architecture integrates inputs, processing, and outputs on the ATmega328P microcontroller, with CAN interfaces via three MCP2515 controllers.

- **Inputs:**

- APPS_5V: Analog pedal signal (pin PC0).
- APPS_3V3: Redundant analog pedal signal (pin PC1).
- Brake: Analog brake signal (pin PC3).
- Start Button: Digital input (pin PC4, active low).

- **Outputs:**

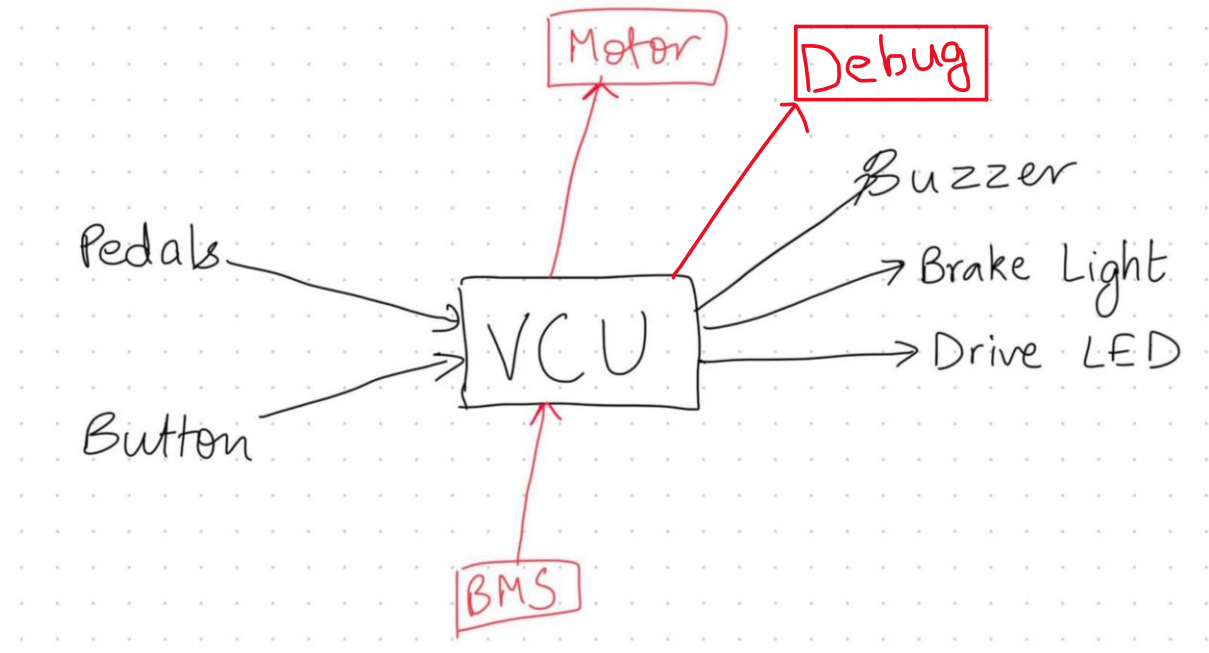
- Brake Light: Digital output (pin PD2, high when brake > threshold).
- Buzzer: Digital output (pin PD4, active during BUZZIN state).
- Drive LED: Digital output (pin PD3, high in DRIVE state).

- **CAN Interfaces:**

- Motor: MCP2515 on PB2 (Arduino pin 10).
- BMS: MCP2515 on PB1 (Arduino pin 9).
- Debug: MCP2515 on PB0 (Arduino pin 8).

All CAN buses operate at 500 kbps with a 20 MHz crystal.

Red lines are for CAN communication and direction of arrows show the direction of data flow.



Logic Flow

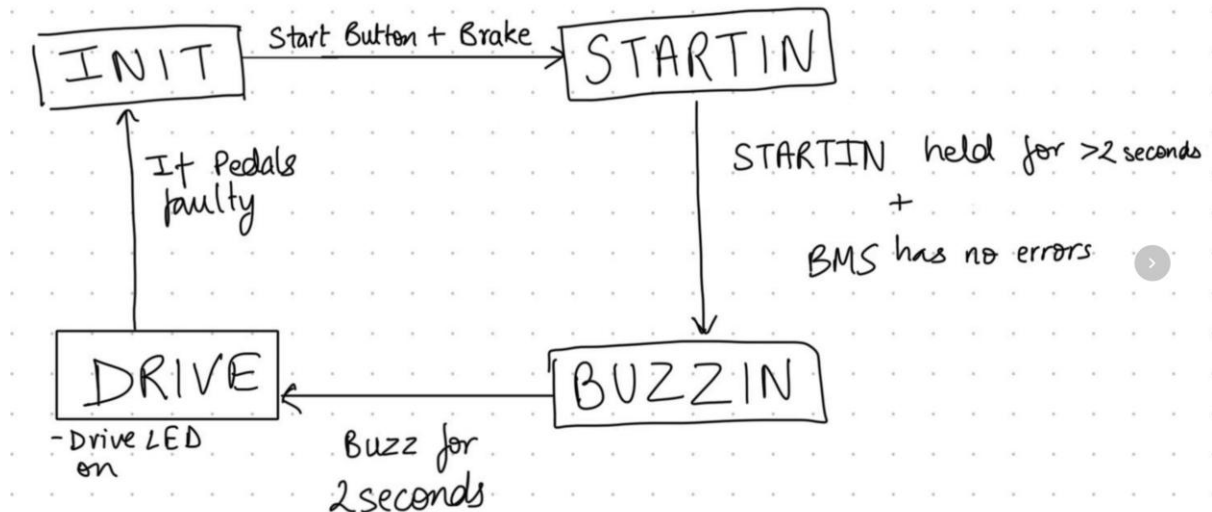
The program uses the Arduino loop() function: Read inputs → Update outputs (e.g., brake light) → Update States → Send CAN messages.

State transitions are managed via an enum (State: INIT=0, STARTIN=1, BUZZIN=2, DRIVE=3):

- **INIT → STARTIN:** Start button pressed and brake > threshold; hold for 2 seconds.

- **STARTIN → BUZZIN:** Conditions held and BMS ready message received.
- **BUZZIN → DRIVE:** After 2-second buzzer activation.
- **Faults** (e.g., APPS difference >10% for >100 ms) reset to INIT from DRIVE.

Fault handling: Continuously monitor scaled APPS values; start timer on detection; reset state if timeout exceeded.



Key Implementations

Pedal Processing

APPS signals are scaled to match ranges ($\text{apps3V3} * 50/33 \approx 1.515 * \text{apps3V3}$), averaged, clamped (0 to PEDAL_MAX - PEDAL_MIN), and linearly mapped to torque (-32768 to 32767). Direction flip via compile-time constant.

After

Fault Detection

Compute absolute difference: $|(\text{apps5V} * 33) - (\text{apps3V3} * 50)| > 10\%$ of range. If true, start fault timer; reset to INIT after 100 ms.

Testing

Pedal validation uses debug CAN messages (ID 0x700 for raw readings, 0x702 for fault diff). Simulate inputs in code or via hardware; verify faults trigger state reset.

CAN Communications

All messages at 500 kbps. Use .DBC file in SavvyCAN for decoding.

- **BMS:** Read ID 0x186040F3; check data[6] == 0x50 for readiness in STARTIN.
- **Motor:** Send ID 0x201, DLC=3; data[0]=0x90, data[1-2]=torque (little-endian int16_t).
- **Debug:**
 - ID 0x700 (pedals): DLC=6; bytes 0-1: apps5V (LE uint16), 2-3: apps3V3, 4-5: brake.
 - ID 0x701 (state): DLC=1; byte 0: State enum (0=INIT, 1=STARTIN, etc.).
 - ID 0x702 (fault): DLC=8 (first 2 bytes used); bytes 0-1: scaled APPS diff (LE uint16).

Maintenance Notes

- **Calibration:** Adjust constants in config.h (e.g., PEDAL_MIN/MAX for ADC range, BRAKE_DEPRESSED_THRESHOLD). Test with actual hardware.
- **Debugging:** Savvy CAN